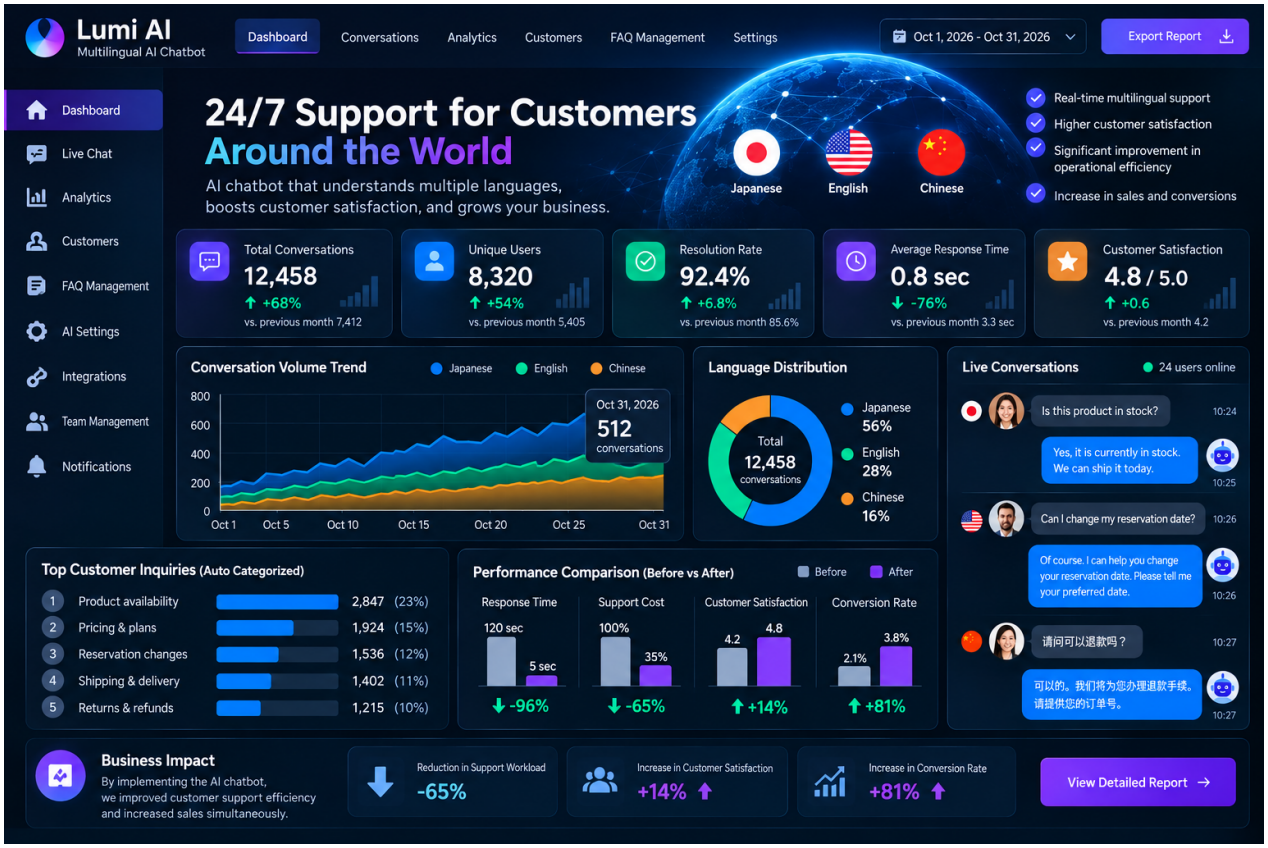
LUMI AI

Multilingual AI Chatbot & Performance Analytics — Portfolio Case Study

A detailed concept for a multilingual customer-support assistant and analytics dashboard, combining AI-assisted conversations, operational metrics, automated inquiry categorization, and business-impact reporting.



Languages	Core components	Suggested stack
Japanese · English · Chinese	Chatbot · Analytics · Live conversation view	Python · FastAPI · OpenAI API · SQL

Important note on metrics: The dashboard image contains illustrative figures (including 92.4% resolution, 0.8-second average response time, and 4.8/5 satisfaction). They are mockup values, not verified production results. Replace them with measurements from a deployed system before presenting them as actual outcomes.

1. Executive Summary

Lumi AI is a proposed multilingual customer-support solution that pairs a conversational assistant with an operational analytics dashboard. The chatbot is intended to answer routine questions, classify incoming inquiries, preserve relevant conversation context, and route requests to a human agent when automation is not appropriate.

The dashboard provides a unified view of usage and service quality. It brings conversation volume, unique users, resolution rate, response time, satisfaction feedback, language distribution, frequent inquiry categories, and live conversations into one interface.

Primary objectives

- **Automate routine support:** handle frequently asked questions and common service requests with consistent responses.
- **Support multiple languages:** respond in Japanese, English, and Chinese, with language detection or explicit language selection.
- **Improve visibility:** provide a clear picture of demand, response performance, resolution outcomes, and common customer needs.
- **Enable informed decisions:** help managers identify high-volume inquiry types and prioritize future automation.
- **Keep human support available:** provide a controlled escalation route for uncertain, sensitive, or account-specific requests.

Intended use cases

- E-commerce product availability, shipping, returns, and order-status inquiries.
- Reservation changes and service information for hospitality or booking businesses.
- SaaS product guidance, account questions, and basic troubleshooting.
- General customer service for businesses serving Japanese-, English-, and Chinese-speaking customers.

2. Product Scope & User Experience

The concept contains two connected experiences: the customer-facing chat interaction and an internal dashboard used by support staff or managers. The customer interface should remain simple and conversational, while the dashboard exposes the detail needed to monitor performance and investigate issues.

Customer-facing chatbot

- **Language handling:** detect the user's language where feasible and allow the user to switch languages.
- **Intent recognition:** classify questions into supported categories such as product availability, pricing, shipping, reservations, and returns.
- **Context management:** retain relevant conversation context within a session while avoiding unnecessary retention of sensitive data.
- **Grounded responses:** use approved knowledge-base content or connected business data for company-specific answers.
- **Fallback and escalation:** explain when an answer cannot be verified and offer a human handoff where available.

Internal dashboard

- **Summary KPI cards:** conversation count, unique users, resolution rate, average response time, and satisfaction.
- **Trend analysis:** daily or weekly conversation volume, split by language or channel.
- **Language distribution:** compare demand across supported languages.
- **Inquiry categorization:** rank common topics to show where automation has the greatest potential.
- **Live conversation feed:** review recent conversations with timestamps and language indicators, subject to access controls.
- **Performance comparison:** compare a defined baseline with post-deployment data when reliable baseline measurements exist.

3. Dashboard Metrics & Definitions

A dashboard is only useful when each metric has a clear, consistent definition. The following definitions are recommended for an implemented version. The values displayed in the concept image are illustrative and should not be used as evidence of actual performance.

Metric	Recommended definition	Implementation note
Total conversations	Count of conversation sessions created within the selected period.	Define when a session starts and ends.
Unique users	Distinct user identifiers active during the period.	Use privacy-aware identifiers; avoid counting raw names.
Resolution rate	Share of eligible conversations resolved without human intervention.	Define 'resolved' and exclude test or abandoned sessions.
Average response time	Average elapsed time from user message to chatbot response.	Report median and percentiles as well as average where possible.
Customer satisfaction	Average score from post-conversation feedback responses.	Display response count and collection method.
Language distribution	Share of eligible conversations by detected or selected language.	Specify treatment for mixed-language conversations.
Conversion rate	Share of eligible sessions resulting in a defined business conversion.	Attribute conversions consistently and state the window.

Reporting recommendations

- Allow date-range selection and compare equivalent periods.
- Show sample size alongside percentages and satisfaction scores.
- Distinguish measured results from estimated or modelled savings.
- Track failed responses, escalations, and unresolved conversations—not only positive outcomes.
- Record metric definitions so reports remain comparable over time.

4. Conversation Flow & Escalation Logic

A typical conversation begins with language identification, followed by intent classification and retrieval of relevant approved information. The assistant then composes a response, checks whether the request has been addressed, and either continues the conversation or offers escalation.

Recommended flow

Step	Processing	Expected behavior
1. Receive	Accept a user message and session context.	Validate payload and associate it with a session.
2. Detect language	Determine or confirm the response language.	Continue in Japanese, English, or Chinese as appropriate.
3. Classify intent	Identify the inquiry category.	Route to FAQ, product lookup, booking, or another handler.
4. Retrieve context	Fetch approved content or authorized business data.	Avoid unsupported claims about orders or availability.
5. Generate answer	Compose a concise, context-aware reply.	Use the correct language and explain limitations.
6. Evaluate	Check for unresolved intent or low confidence.	Ask a clarifying question or escalate if needed.
7. Log outcome	Record minimal operational metadata.	Update dashboard metrics without exposing unnecessary PII.

Escalation triggers

- The user explicitly asks to speak with a person.
- The answer depends on private account data that the chatbot cannot securely access.
- The request is outside the supported knowledge base or the assistant cannot verify the answer.
- The conversation involves a complaint or sensitive issue requiring human judgment.
- A backend or external API fails and the requested action cannot be confirmed.

5. Recommended Technical Architecture

Python and FastAPI are suitable for a lightweight API backend that receives chat messages, orchestrates AI calls, retrieves business context, and records operational events. A relational database can store conversation metadata and feedback, while a separate analytics layer aggregates metrics for the dashboard.

Layer	Suggested technology	Responsibilities
Web interface	HTML, CSS, JavaScript or a frontend framework	Chat UI, dashboard charts, filters, accessibility, and session display.
API backend	Python + FastAPI	Authentication, request validation, chat orchestration, and API endpoints.
AI layer	OpenAI API	Language-aware response generation, classification, and summarization.
Knowledge retrieval	Approved FAQ store; optional search/vector retrieval	Retrieve relevant product or policy information before answer generation.
Data storage	PostgreSQL or another SQL database	Sessions, event metadata, feedback, and aggregated reporting data.
Integration layer	REST APIs and webhooks	Connect CRM, booking, order, ticketing, or messaging services as required.
Monitoring	Structured logs and error tracking	Observe latency, API errors, fallbacks, and service health.

Example API responsibilities

- **POST /chat:** accept a message, validate session context, and return a chatbot response.
- **GET /analytics/summary:** return aggregated KPI values for a selected date range.
- **GET /analytics/trends:** return time-series conversation and response metrics.
- **GET /conversations:** return authorized conversation summaries for support staff.
- **POST /feedback:** record a satisfaction rating or resolution feedback.

6. Data Model & Analytics Pipeline

A reliable dashboard should be powered by defined event records rather than manually entered KPI values. Each event should be timestamped, associated with a conversation identifier, and limited to the data needed for service operation and analysis.

Suggested core entities

- **Conversation:** conversation ID, pseudonymous user ID, start/end time, language, channel, and status.
- **Message event:** conversation ID, timestamp, sender type, latency, and safe metadata about processing outcome.
- **Intent classification:** category, confidence where available, and routing outcome.
- **Resolution event:** whether the request was resolved, escalated, abandoned, or left unresolved, with a defined rule.
- **Feedback:** rating, timestamp, optional category, and a reference to the relevant conversation.
- **Business conversion:** a defined event such as completed booking or purchase, linked through a privacy-aware attribution method.

Analytics pipeline

- Collect events from the chat API and connected systems.
- Validate event shape and normalize timestamps and categories.
- Store raw operational events with controlled retention.
- Aggregate metrics by date, language, intent, channel, and outcome.
- Serve summarized data to the dashboard through authenticated API endpoints.
- Reconcile dashboard metrics against source records during testing.

7. Security, Privacy & Reliability

A customer-support chatbot may handle personal information or account-specific questions. The design should minimize stored data, restrict dashboard access, and avoid claiming successful actions unless the connected system confirms them.

Security and privacy controls

- Use authenticated and role-based access to internal dashboards and conversation records.
- Keep API keys in environment variables or a managed secrets store.
- Avoid sending unnecessary personal or sensitive data to the AI service.
- Define retention and deletion policies for conversation logs and analytics events.
- Protect endpoints with validation, rate limits, and appropriate abuse controls.
- Use TLS for data in transit and appropriate database access controls.
- Provide a clear privacy notice and follow applicable legal requirements for the deployment region.

Reliability and quality checks

- Test supported languages, mixed-language messages, and ambiguous questions.
- Measure latency at median and higher percentiles, not only average response time.
- Test missing knowledge, conflicting information, and unavailable external APIs.
- Ensure failed order, reservation, or account actions are not reported as successful.
- Monitor fallback and escalation rates alongside resolution rate.
- Evaluate answer quality with representative test conversations and human review.

8. Testing & Evaluation Plan

The evaluation process should establish a baseline, test the chatbot against representative user needs, and report results with clear definitions and sample sizes.

Recommended test suites

- **Functional tests:** verify session creation, message handling, language selection, logging, and dashboard queries.
- **Language tests:** evaluate Japanese, English, and Chinese responses for meaning, tone, and consistency.
- **Grounding tests:** check that company-specific responses reflect approved source content and avoid fabricated facts.
- **Routing tests:** verify each intent reaches the correct handler or human escalation path.
- **Load tests:** measure latency and error rates at expected concurrency levels.
- **Security tests:** check authorization boundaries, input validation, rate limits, and data exposure.
- **Analytics reconciliation:** compare dashboard totals with underlying conversation events.
- **Failure tests:** simulate timeouts, rate limits, malformed model output, and unavailable business APIs.

Suggested acceptance criteria

- All core chat and dashboard paths pass the agreed test suite.
- The system returns a clear fallback when it cannot answer reliably.
- No private conversation is exposed to an unauthorized dashboard user.
- Metric definitions are documented and dashboard figures reconcile to stored events.
- Response-time and availability targets are agreed before production launch.
- Performance claims are published only after collecting reproducible deployment data.

9. Implementation Roadmap

Phase	Activities	Deliverable
1. Requirements	Define languages, intents, data sources, user roles, and KPI rules.	Scope and metric definitions.
2. Conversation design	Design supported flows, fallback messages, and escalation rules.	Conversation specification.
3. Backend & AI	Implement API, model calls, validation, and context retrieval.	Chat service prototype.
4. Data & analytics	Store events and build aggregation endpoints.	Analytics data layer.
5. Dashboard UI	Build KPI cards, charts, filters, and conversation feed.	Dashboard prototype.
6. Integrations	Connect agreed CRM, booking, order, or support services.	Integrated workflows.
7. QA & deployment	Test security, language quality, latency, and data accuracy.	Tested release and documentation.

Potential future enhancements

- Additional languages and channels such as web chat, messaging apps, or email.
- Retrieval-augmented generation over approved company documentation.
- Agent handoff with conversation summaries and ticket creation.
- CRM and booking integrations with permission-aware actions.
- A/B testing of prompts and support flows.
- Scheduled reports and alerts for unusual changes in volume or service quality.

10. Portfolio Summary & Metadata

Lumi AI is a detailed concept for a multilingual chatbot and analytics platform. It brings together language-aware support, intent classification, grounded answer generation, escalation, operational event tracking, and a dashboard for reviewing service quality and business outcomes.

Suggested Workana metadata

Field	Suggested value
Project title	AI Chatbot | Multilingual Customer Support & Performance Analytics Dashboard
Technology	Python, FastAPI, OpenAI API, SQL, HTML, CSS, JavaScript
Skills	Python, Artificial Intelligence, API Integration, Chatbot Development, Data Analysis
Core capabilities	Multilingual chat, automated support, analytics, API integration, reporting
Primary visual	Lumi AI Chatbot Performance Analytics Dashboard

Portfolio integrity note: The included dashboard is a visual concept, and the displayed KPI values are illustrative mock data. Unless the chatbot has been implemented and evaluated, describe this as a design and technical architecture concept. For a live project, replace illustrative metrics with measured results and document the evaluation method.